VIVEKANAND EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY

Department of Computer Engineering

CA: II. Virtual Lab: Miniproject

Topic: Implementation of String Matching Visualiser

Under the subject: Design Analysis Of Algorithm (DAA)

Year: S.E. Semester : IV

Submitted by

Bhagyashri Amit Joshi (D7B-23)

Under the guidance of

Prof. Vidya Zope

Subject Teacher

(2025-2026)

Index

Title	Page no
Chapter 1: Introduction	1
Chapter 2: Concept, Algorithm, and Data Structures used	4
Chapter 3: Code (commented) with intermediate steps	7
Chapter 4: Visualisation and UI Design	12
Chapter 5: Complexity Analysis	16
Chapter 6: Conclusion	19
REFERENCES: Books, Youtube links, Research Papers	21

Chapter 1: Introduction

1.1 Objective:

The primary objective of this laboratory project is to design and implement an interactive String Matching Algorithm Visualizer using Java Swing. The visualizer demonstrates the internal step-by-step execution of three classical pattern-searching algorithms: Naive (Brute Force), Knuth-Morris-Pratt (KMP), and Rabin-Karp: helping students develop a concrete, visual understanding of how these algorithms compare characters, handle mismatches, and locate a pattern within a text string.[1], [2]

1.2 Problem Statement:

Given a text string (haystack) and a pattern string (needle), the problem of string matching requires finding all positions within the text where the pattern occurs. This is one of the most fundamental operations in computer science, underpinning applications from text editors (Find & Replace) to DNA sequence analysis, intrusion detection systems, and search engines.[1]
Formally, given:

- Text T of length n: $T = T[0], T[1], \ldots, T[n-1]$
- Pattern P of length m: $P = P[0], P[1], \ldots, P[m-1]$
- Condition: $m \leq n$

The goal is to find all valid shifts s such that:
$T[s \,..\, s+m-1] == P[0 \,..\, m-1]$ for some $0 \leq s \leq n-m$ [1]

1.3 Scope of the Visualizer:

This project provides the following capabilities:

- Interactive input for any text and pattern (uppercase alphabetic, max 40 characters)
- Three selectable algorithms: Naive, KMP, and Rabin-Karp
- Animated, step-by-step character comparison with color-coded cell.
- Pause, Resume, and Reset controls, plus an adjustable speed slider
- A live comparison counter and a bar chart showing algorithm efficiency
- A KMP-specific LPS (Longest Prefix Suffix) array display table
- An algorithm explanation panel for viva and lab examination support

1.4 Packages and Methods Learned in This Lab:

This project provided hands-on exposure to a wide range of Java Swing and AWT classes. The table below summarizes every major class and method used, the package it belongs to, and its specific role in the visualizer.

Class / Method	Package	Role in this Project
JFrame	javax.swing	Top-level application window container
JPanel	javax.swing	General-purpose container; used as drawing canvas
JButton	javax.swing	Clickable button for Start, Reset, Next Step
JLabel	javax.swing	Display static text: edge weights, status messages
JComboBox	javax.swing	Dropdown: algorithm selection
JSlider	javax.swing	Speed control slider for animation
JTextField	javax.swing	Editable text input for custom graph data
JTextArea	javax.swing	Multi-line text: algorithm explanation panel
JScrollPane	javax.swing	Wraps components to add scroll capability
javax.swing.Timer	javax.swing	Fires events at fixed intervals: drives animation
Graphics2D	java.awt	2D rendering: draw vertices (ovals), edges (lines)
paintComponent()	java.awt	Override to perform custom drawing on a JPanel
GridBagLayout	java.awt	Flexible grid layout manager for input panel
BorderLayout	java.awt	Divides container into NORTH/CENTER/SOUTH zones
Color	java.awt	Create custom RGB colors for vertex/edge highlights
BasicStroke	java.awt	Set line width and dash style for edge drawing
RenderingHints	java.awt	Enable anti-aliasing for smooth circles and lines
SwingUtilities.invokeLater()	javax.swing	Thread-safe GUI initialization on EDT

1.4.1 Key Swing Concepts Applied:

1. Event Dispatch Thread (EDT): All Swing operations are not thread-safe. SwingUtilities.invokeLater() guarantees that GUI creation and updates happen on the EDT, preventing race conditions.

2. Custom Painting with paintComponent(): Rather than placing individual JLabels for each character, the VisualizationPanel overrides paintComponent(Graphics g) to draw character cells using Graphics2D. This allows pixel-precise control over position, color, size, and borders of each cell.

3. javax.swing.Timer for Animation: The Timer fires an ActionEvent on the EDT at a configurable delay (50ms–1000ms). Each tick advances the algorithm by one comparison step and calls repaint(), which triggers paintComponent() with the new state.

4. Graphics2D Rendering Pipeline: Anti-aliasing hints (RenderingHints.KEY_ANTIALIASING) are applied to smooth cell borders and text. fillRoundRect() and drawRoundRect() draw the character cells, and drawString() centers characters within each cell using FontMetrics.

Chapter 2: Concept, Algorithm, and Data Structures

2.1 Naive (Brute Force) Algorithm:

The Naive algorithm is the most straightforward approach to pattern matching. It systematically tries every possible alignment of the pattern against the text, comparing characters one by one. On a mismatch, it shifts the pattern one position to the right and restarts from the first character of the pattern. [1], [2]

2.1.1 Algorithm Pseudocode:

```
NAIVE-SEARCH(text T[0..n-1], pattern P[0..m-1])
  for i = 0 to n - m:
        j = 0
        while j < m and T[i+j] == P[j]:
        j = j + 1
        if j == m:
        print 'Pattern found at index', i
```

Time Complexity: $O((n - m + 1) \times m)$: Worst case: $O(n \times m)$ [1]
Space Complexity: $O(1)$: No extra data structure needed
Weakness: When a partial match of length k is followed by a mismatch, the algorithm discards all k matched characters and re-examines them from scratch at the next shift. This is wasteful: KMP eliminates this redundancy.

2.2 KMP (Knuth-Morris-Pratt) Algorithm:

KMP is a linear-time pattern matching algorithm published in 1977 by Knuth, Morris, and Pratt. Its key insight is that on a mismatch after j characters of the pattern have matched, the algorithm does not need to re-examine the j characters of the text. It uses a precomputed LPS (Longest Proper Prefix which is also a Suffix) array to determine the next alignment. [3]

2.2.1 LPS (Failure Function) Array:

The LPS array for a pattern P of length m is defined as:
lps[i] = length of the longest proper prefix of P[0..i]
that is also a suffix of P[0..i] [3], [7]

Example: Pattern: ABABCABABD

Row	0	1	2	3	4	5	6	7	8	9
Pattern Char	A	B	A	B	C	A	B	A	B	D
LPS Value	0	0	1	2	0	1	2	3	4	0

Reading the table: lps[7] = 3 means the substring "ABABCABA" has "ABA" as both its longest proper prefix and suffix of length 3.

2.2.2 KMP Search Algorithm:

```
KMP-SEARCH(text T[0..n-1], pattern P[0..m-1], lps[0..m-1])
  i = 0   // text pointer
  j = 0   // pattern pointer
  while i < n:
        if T[i] == P[j]:
        i++, j++
        if j == m:              // full match
        print 'Found at index', i - j
        j = lps[j - 1]          // continue searching
        else:
        if j != 0:
        j = lps[j - 1]          // KEY: skip already-matched prefix
        else:
        i++                     // no match possible here
```

Time Complexity: O(n + m) : preprocessing O(m) + search O(n) [3], [5]
Space Complexity: O(m) : for the LPS array

2.3 Rabin-Karp Algorithm:

Rabin-Karp, published by Rabin and Karp in 1987, uses hashing to speed up pattern matching. Instead of comparing characters directly at each position, it computes a hash of the pattern and a rolling hash of each text window. Only when the hashes match does it perform a character-by-character verification (to guard against collisions). [4], [6]

2.3.1 Rolling Hash Formula:

Hash of string S[0..m-1] with base d and prime q:

h = (S[0]*d^(m-1) + S[1]*d^(m-2) + ... + S[m-1]) mod q

Rolling update (remove S[i], add S[i+m]):

h_new = (d * (h_old - S[i] * d^(m-1)) + S[i+m]) mod q

Constants used in this project:

d = 256 (ASCII alphabet size)

q = 101 (prime: minimizes false positives)

Time Complexity: O(n + m) average | O(nm) worst case (all hash collisions) [4]

Space Complexity: O(1) : only a few integer variables

Best use case: Multiple pattern search (search for k patterns simultaneously in O(n + km))

2.4 Data Structures Used:

2.4.1 AlgorithmStep (Custom Record Class):

A plain Java class acting as a data transfer object (DTO). Each instance captures the complete state of one comparison moment:

- patternOffset: the current window start position in text
- patternCompareIndex: which pattern character is being compared
- textCompareIndices: corresponding text position(s)
- isMatch: boolean: match (green) or mismatch (red)
- foundAt: set to ≥ 0 when full pattern is found; -1 otherwise

All three algorithms produce a List<AlgorithmStep>. The VisualizationPanel reads this list sequentially, one step per Timer tick, without knowing which algorithm generated it. This is the Strategy / Iterator design pattern in practice. [2]

2.4.2 ArrayList<AlgorithmStep>:

Java's ArrayList is used to collect steps during the algorithm run. Random access (get(i)) is O(1), which is important because the Timer must retrieve step i efficiently on each tick. The list is pre-populated before animation starts, so the animation itself has zero algorithmic overhead.

2.4.3 int[ ] LPS Array (KMP):

A primitive integer array of size m (pattern length) holds the failure function values. It is computed once during KMP preprocessing in O(m) time and displayed in the LPS table panel. The array is also used internally during the KMP search to determine the jump distance on a mismatch.

Chapter 3: Code with Intermediate Steps

This chapter presents the core algorithmic code with detailed comments explaining every step. The complete project contains 12 Java source files across three packages (main, ui, algorithms). Below, the most critical sections are annotated in depth.

3.1 AlgorithmStep.java: Data Class:

```
package algorithms;
/**
 * Represents a single comparison moment in the algorithm.
 * All three algorithms produce List<AlgorithmStep>.
 */
public class AlgorithmStep {
        public int patternOffset;        // where pattern window starts in text
        public int patternCompareIndex; // which pattern char is being compared
        public int[] textCompareIndices;// corresponding text positions
        public boolean isMatch;        // true = green, false = red
        public int foundAt;            // >= 0 if full match found, else -1
        public AlgorithmStep(int offset, int patIdx, boolean match, int found) {
    this.patternOffset          = offset;
    this.patternCompareIndex = patIdx;
        this.isMatch            = match;
        this.foundAt            = found;
        // Derived: text position = offset + pattern index
    this.textCompareIndices  = new int[]{ offset + patIdx };
        }
}
```

3.2 NaiveAlgorithm.java: Brute Force:

```
public static List<AlgorithmStep> run(String text, String pattern) {
   List<AlgorithmStep> steps = new ArrayList<>();
        int n = text.length(), m = pattern.length();
        // OUTER LOOP: slide pattern window from position 0 to (n-m)
        for (int i = 0; i <= n - m; i++) {
        // INNER LOOP: compare each character of pattern
        for (int j = 0; j < m; j++) {
        boolean match = (text.charAt(i+j) == pattern.charAt(j));
```

```
        if (match && j == m - 1) {
                // Last char also matched → FULL MATCH at position i
            steps.add(new AlgorithmStep(i, j, true, i));
        } else if (!match) {
                // MISMATCH → record step, break inner loop
                // Pattern shifts right by 1 (outer loop i++)
            steps.add(new AlgorithmStep(i, j, false, -1));
                break;
        } else {
                // Partial match: record and continue inner loop
            steps.add(new AlgorithmStep(i, j, true, -1));}}}
        return steps;
}
```

3.3 KMPAlgorithm.java: Find + Union Concept via LPS:

The computeLPS() method is the preprocessing phase. It uses a two-pointer technique that is analogous to running KMP on the pattern against itself. The 'find' operation here is finding the longest prefix that is also a suffix; the 'union' is extending that match length. [3]

```
// PHASE 1: Compute LPS arraypublic static int[] computeLPS(String pattern) {
        int m = pattern.length();
        int[] lps = new int[m];
        lps[0] = 0;       // base case: single char has no proper prefix
        int len = 0;   // length of current matching prefix
        int i   = 1;   // scan from second character
        while (i < m) {
        if (pattern.charAt(i) == pattern.charAt(len)) {
        // PREFIX EXTENSION: match found, extend length
        len++;
        lps[i] = len;
        i++;
        } else {
        if (len != 0) {
                // FALLBACK (like 'find' in Union-Find):
                // Use lps[len-1] to try a shorter prefix
                // This avoids re-scanning from scratch
                len = lps[len - 1];
        } else {
                lps[i] = 0;  // no prefix possible
```

```
                i++;}}}
        return lps;
}

// PHASE 2: KMP Search using LPS
public static List<AlgorithmStep> run(String text, String pattern) {
        int[] lps = computeLPS(pattern);
        int n = text.length(), m = pattern.length();
        int i = 0, j = 0;           // i = text ptr, j = pattern ptr
        while (i < n) {
        boolean match = (text.charAt(i) == pattern.charAt(j));
        int offset = i - j; // current window start
        if (match) {
        if (j == m - 1) {
                // FULL MATCH: record, then jump using LPS
           steps.add(new AlgorithmStep(offset, j, true, offset));
                j = lps[j - 1]; // skip redundant re-checks
        } else {
           steps.add(new AlgorithmStep(offset, j, true, -1));
                i++; j++;
        }
        } else {
        steps.add(new AlgorithmStep(offset, j, false, -1));
        if (j != 0)
                j = lps[j - 1]; // KEY SKIP: text pointer stays!
        else
                i++;                // no match possible, move forward
   }}
        return steps;
}
```

3.4 RabinKarpAlgorithm.java: Rolling Hash:

```
public static List<AlgorithmStep> run(String text, String pattern) {
        int n = text.length(), m = pattern.length();
        int D = 256, Q = 101;   // alphabet size, prime modulus
        // h = D^(m-1) mod Q - used to remove the leading character
        int h = 1;
        for (int k = 0; k < m - 1; k++) h = (h * D) % Q;
        // Compute initial hash of pattern and first text window
```

```
        int pH = 0, wH = 0;
        for (int k = 0; k < m; k++) {
        pH = (D * pH + pattern.charAt(k)) % Q;
        wH = (D * wH  + text.charAt(k))   % Q;
        }
        for (int i = 0; i <= n - m; i++) {
        if (wH == pH) {
        // Hash match: verify character-by-character
        // (guards against hash collisions)
        for (int j = 0; j < m; j++) {
                boolean cm = text.charAt(i+j) == pattern.charAt(j);
           steps.add(new AlgorithmStep(i, j, cm,
                  (cm && j==m-1) ? i : -1));
                if (!cm) break;
        }
        } else {
        // Hash mismatch: skip without comparing (O(1) step)
        steps.add(new AlgorithmStep(i, 0, false, -1));
        }
        // ROLLING HASH UPDATE: remove text[i], add text[i+m]
        if (i < n - m) {
        wH = (D * (wH - text.charAt(i) * h) + text.charAt(i+m)) % Q;
        if (wH < 0) wH += Q; // handle Java's negative modulo
        }
        }
        return steps;
} [4]
```

3.5 VisualizationPanel: paintComponent() Drawing Logic:

The heart of the UI. This method is called automatically by Swing whenever repaint() is triggered. It draws the entire animation frame from scratch each tick.

```
@Override
protected void paintComponent(Graphics g) {
   super.paintComponent(g);  // clear background
        Graphics2D g2 = (Graphics2D) g;
        // Enable anti-aliasing for smooth rendering
   g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
                RenderingHints.VALUE_ANTIALIAS_ON);
```

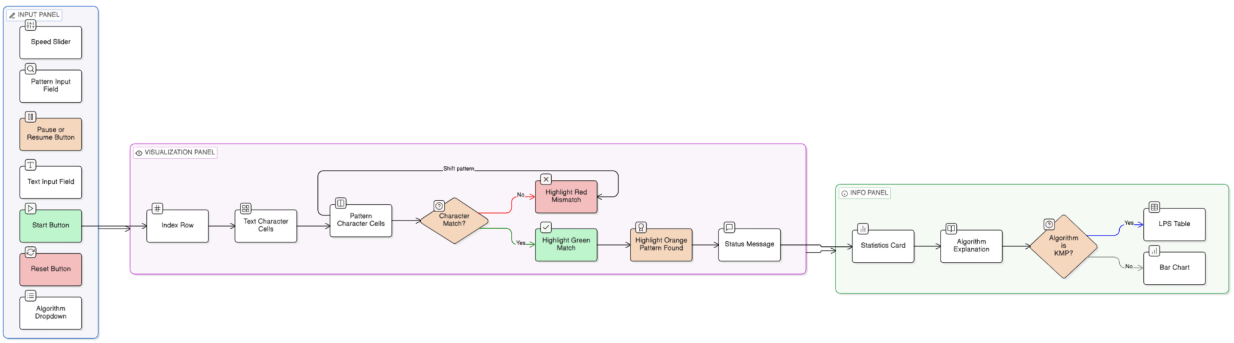
```
        // Calculate x-offset to center the text row
        int totalWidth = text.length() * (CELL_SIZE + CELL_GAP) - CELL_GAP;
        int startX        = (getWidth() - totalWidth) / 2;
        drawIndexRow(g2, startX);   // 0 1 2 3 ... above text cells
        drawTextRow(g2,  startX);   // text cells with highlight colors
        if (activeStep != null) {
        drawPatternRow(g2, startX); // pattern shifted by patternOffset
      drawStatusMessage(g2);            // match/mismatch message at bottom
        }
}
// Draw one character cell (rounded rectangle + centered letter)
private void drawCell(Graphics2D g2, int x, int y,
                String ch, Color bg, Color border) {
        g2.setColor(bg);
        g2.fillRoundRect(x, y, CELL_SIZE, CELL_SIZE, 6, 6);
        g2.setColor(border);
        g2.setStroke(new BasicStroke(1.2f));
        g2.drawRoundRect(x, y, CELL_SIZE, CELL_SIZE, 6, 6);
        // Center character using FontMetrics
        FontMetrics fm = g2.getFontMetrics();
        int cx = x + (CELL_SIZE - fm.stringWidth(ch)) / 2;
        int cy = y + (CELL_SIZE + fm.getAscent() - fm.getDescent()) / 2;
        g2.setColor(new Color(30, 35, 50));
        g2.drawString(ch, cx, cy);
}
```

Chapter 4: Visualization and UI Design

This chapter describes the visual design of the application, explaining the layout, color scheme, and the step-by-step animation logic as it appears on screen.

4.1 UI Layout Overview:

The application window is divided into three horizontal sections:



4.2 Color Coding System:

Every cell in the visualization follows a strict color convention:

Color	State	Meaning
Blue (#64A0F0)	Currently being compared	Neutral: comparison in progress
Green (#50B964)	Character match	T[i] == P[j]: correct so far
Red (#E65046)	Character mismatch	T[i] != P[j]: pattern must shift
Orange (#FFAA1E)	Full pattern found	All m characters matched: FOUND
White (#F5F7FA)	Not yet visited	Default idle state

4.3 Step-by-Step Animation Walkthrough:

Example: Text = "ABABDABACDABABCABAB", Pattern = "ABABC"

Step 1: Initial State (shift = 0)

Pattern ABABC is aligned at position 0. Comparison begins at index 0. Both cells highlighted blue.

Step 2: Partial Match

A=A ✓, B=B ✓, A=A ✓, B=B ✓ (all green). Then A vs D → mismatch (red). Naive shifts right by 1.

Step 3: Pattern Shift

Pattern moves right. Status message shows: '✗ Mismatch at text[4] vs pattern[4]: shift pattern'. Shift counter increments.

Step 4: Full Match Found

At shift = 10, all 5 characters match. All text and pattern cells turn orange. Status shows: '✔ Pattern found at index 10!'

4.4 KMP LPS Table Display:

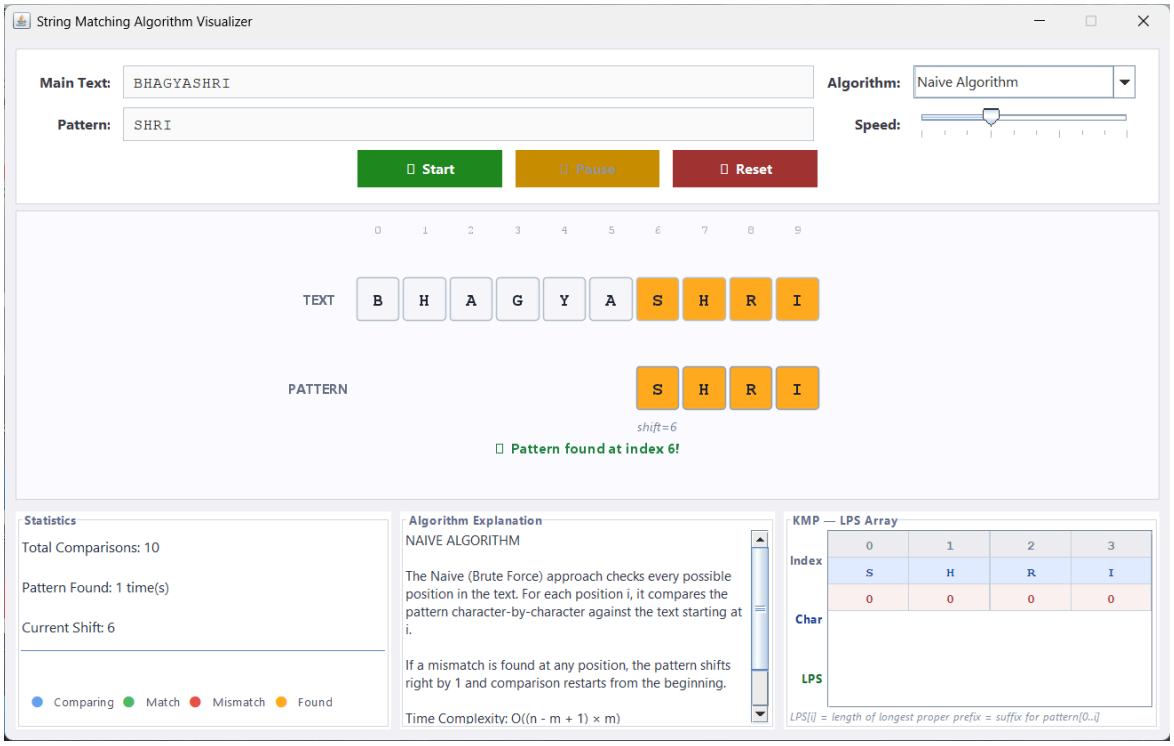
When KMP is selected, the bottom-right panel switches from the bar chart to the LPS table. For pattern ABABC, the table shows index row (0–4), pattern characters (A B A B C), and LPS values (0 0 1 2 0). LPS values > 0 are highlighted in green, 0 values in pale orange, making it easy to identify the prefix-suffix structure at a glance. [8]

4.5 Speed Control:

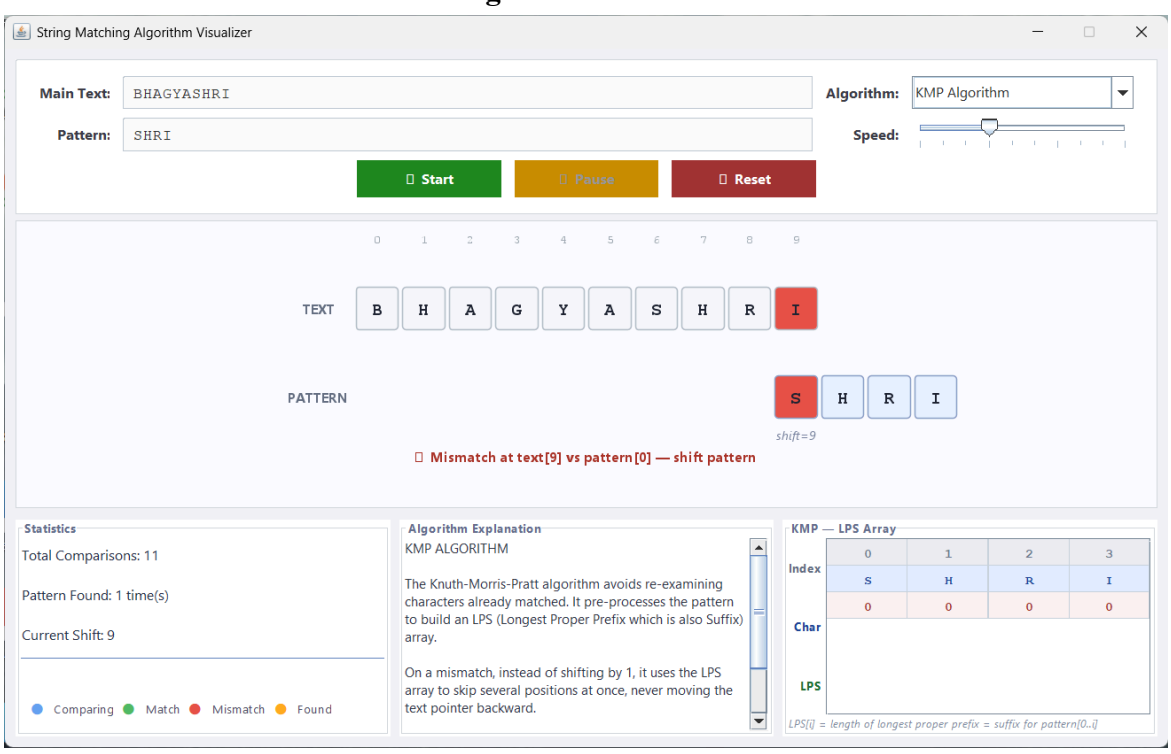
A JSlider with range 1–10 controls the animation delay. Slider value v maps to delay d using the formula $d = 1050 - (v \times 100)$ milliseconds. At v = 1: d = 950ms (very slow, good for demos). At v = 10: d = 50ms (very fast, good for showing algorithm efficiency). The delay is updated in real-time by calling animTimer.setDelay(d) inside the slider's ChangeListener. [8]

4.6 Implementation Screenshots:

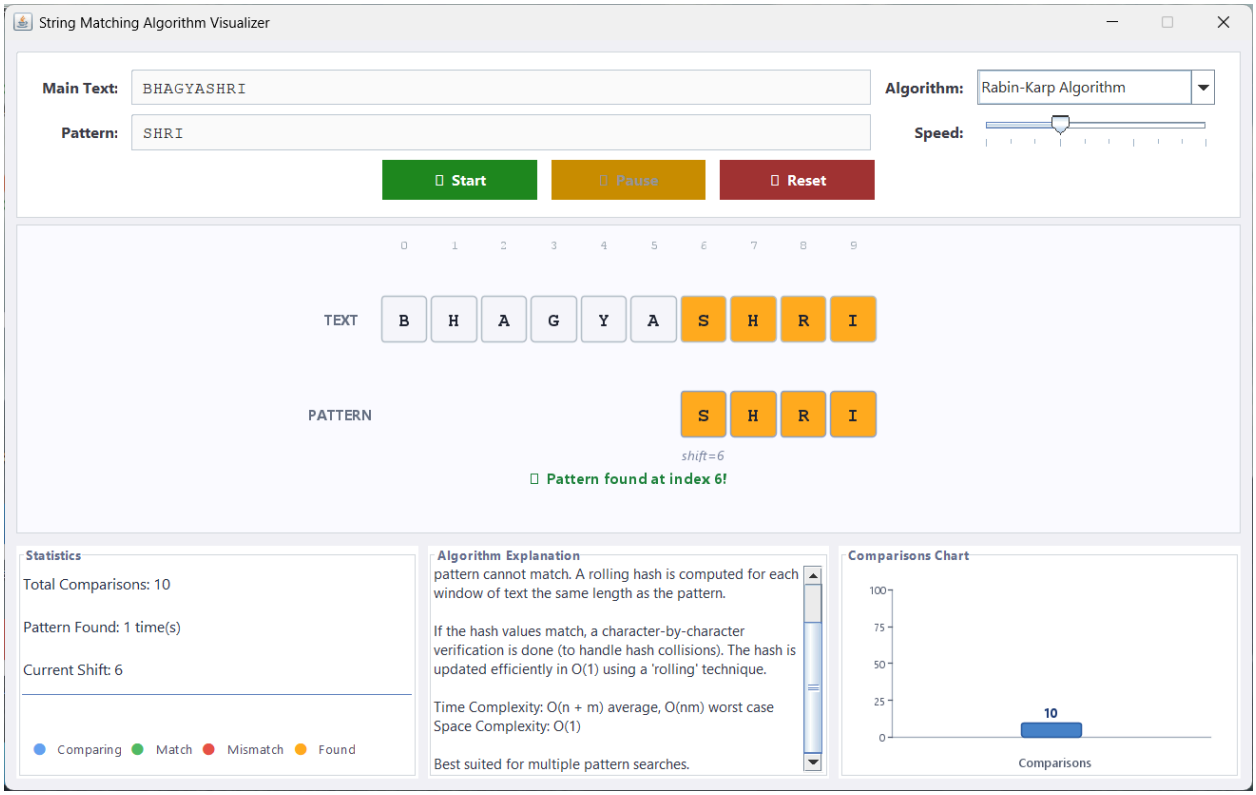
4.6.1 Naive Algorithm:



4.6.2 Knuth-Morris-Pratt Algorithm:



4.6.3 Robin-Karp Algorithm:



Chapter 5: Complexity Analysis

5.1 Time Complexity Summary:

Algorithm	Best Case	Worst Case	Space	Notes
Naive (Brute Force)	O(n)	O(n × m)	O(1)	Simple but re-examines matched characters
KMP	O(n + m)	O(n + m)	O(m)	Linear guaranteed; never moves text pointer back
Rabin-Karp	O(n + m)	O(n × m)	O(1)	Worst case only when all windows cause hash collision

where n = length of text, m = length of pattern. [1], [3], [4]

5.2 Naive Algorithm: Detailed Analysis:

5.2.1 Worst Case: O(n × m):

Occurs when every alignment causes m−1 character matches followed by a mismatch. Classic example: T = "AAAAAAB", P = "AAAB". For each of the (n−m+1) positions, the algorithm matches m−1 = 3 characters before hitting B vs A. Total comparisons ≈ (n−m+1) × m. [1]

5.2.2 Best Case: O(n):

When the first character of the pattern never matches the text (e.g., T = "BBBBB", P = "A..."). Each window position causes exactly 1 comparison before a mismatch, giving O(n) total comparisons.

5.2.3 Why Naive is Inefficient:

On a mismatch after k characters have matched, Naive discards all k matched characters and restarts comparison from pattern index 0 at the next shift. It has no memory of what it already verified. This is precisely the redundancy that KMP eliminates.

5.3 KMP Algorithm: Detailed Analysis:

5.3.1 Why O(n + m) is Guaranteed:

The text pointer i never moves backward. Each iteration of the while loop either increments i (on a character match) or keeps i fixed while j decreases via the LPS fallback. Since j can decrease at most as many times as it has increased, and it increases at most n times total, the search phase runs in O(n). The preprocessing phase (computing the LPS array) runs in O(m) by the same argument applied to the pattern against itself. Total: O(n + m). [3]

5.3.2 Comparison Count Demonstration:

For Text = "AAAAAB" (n = 6) and Pattern = "AAAB" (m = 4):

- Naive: $(6 - 4 + 1) \times 4 = 12$ comparisons
- KMP: $6 + 4 = 10$ comparisons: and critically, this bound holds regardless of the pattern structure

5.3.3 Role of the LPS Array:

The LPS array is what makes the skip possible. For pattern "AAAB", lps = [0, 1, 2, 0]. On a mismatch at pattern index j = 3 (the 'B'), KMP jumps to j = lps[2] = 2 instead of j = 0. This means the two already-matched 'A' characters are not re-examined: the pattern window effectively shifts by only 1 position while the text pointer stays in place.

5.4 Rabin-Karp: Hash Collision Analysis:

5.4.1 Average Case: O(n + m):

The rolling hash reduces most positions to a single O(1) hash comparison. Only when the hash of a text window equals the hash of the pattern does the algorithm perform a character-by-character verification. In the average case with a good hash function, very few windows trigger verification, keeping the total close to O(n + m). [4]

5.4.2 Collision Probability:

The probability of a spurious hash match (collision) for a random string of length m over prime q is approximately m/q. With q = 101 and m = 5 in this project, this is ≈ 5% ; meaning on average 5% of windows require full character verification even without a true match.

5.4.3 Worst Case: O(n × m):

Occurs when every window produces a hash collision (e.g., all characters identical). Every window triggers O(m) verification, degrading to Naive performance. This is avoided in practice by choosing a larger prime q.

5.5 Space Complexity:

- Naive: O(1): no additional storage beyond the input strings themselves
- KMP: O(m): one integer array of size m for the LPS (failure function) values
- Rabin-Karp: O(1): only a handful of integer variables (hash values, the precomputed h constant)
- Visualization overhead: O(steps): the List<AlgorithmStep> pre-collects every comparison before animation begins; in the worst case (Naive on "AAAAAAB" / "AAAB") this reaches O(n × m) entries

5.6 Efficiency Visible in the Visualizer:

The comparison counter in the Statistics panel makes efficiency differences immediately visible. Running all three algorithms on T = "AAAAAB", P = "AAAB" produces the following observable differences:

- Naive shows 12 comparison flashes, with repeated red mismatches at position 3 every time the 'B' is reached. The pattern visibly resets to index 0 after every mismatch.
- KMP shows 10 comparisons. The LPS table in the bottom-right panel highlights lps[2] = 2, showing the algorithm jumping j from 3 to 2 instead of 0. The text pointer never moves left: visible because the index highlight on the text row only ever moves right.
- Rabin-Karp shows most windows as a single immediate red flash (hash mismatch: O(1) skip), with character-level comparison only triggered when hash values happen to collide. The bar chart grows slowly compared to Naive, making the efficiency advantage visually clear.

Chapter 6: Conclusion

6.1 Summary of Work Done:

This laboratory project successfully implements and visualizes three classical string matching algorithms: Naive, KMP, and Rabin-Karp: as an interactive Java Swing application. The following deliverables were completed:

- Designed and implemented a fully interactive GUI with 12 Java source files across three packages. [8], [9], [10], [12]
- Built a custom animation engine using javax.swing.Timer and Graphics2D that displays character-level comparisons step by step. [8]
- Implemented all three algorithms from scratch, producing List<AlgorithmStep> output that decouples algorithm logic from the visualization layer. [9], [10], [12]
- Added a real-time LPS table panel specific to KMP, showing how the failure function is computed. [9], [10], [12]
- Integrated a live bar chart drawn with Graphics2D (no external library) showing cumulative comparison counts. [8]
- Provided pause/resume controls and a speed slider for flexible demonstration. [9], [10], [12]

6.2 Learning Outcomes:

- Gained practical experience with Java Swing component hierarchy, EDT, and custom rendering via paintComponent().
- Understood why KMP's O(n+m) guarantee holds: the text pointer never moves backward.
- Learned rolling hash construction and why prime modulus q = 101 reduces false positives.
- Applied the Strategy pattern: three algorithm classes share the same AlgorithmStep output contract, making the visualizer algorithm-agnostic.
- Experienced the full software development cycle: design, implementation, testing, and documentation.

6.3 Comparison and Recommendations:

For general-purpose single-pattern searching, KMP is the recommended choice: it provides a linear-time guarantee with no worst-case degradation. Naive is acceptable for short texts ($n \leq 100$) or when simplicity is paramount. Rabin-Karp is best suited for multi-pattern search (e.g., plagiarism detection, DNA fingerprinting) where its $O(km)$ preprocessing for k patterns amortizes well. [3], [4]

6.4 Future Enhancements:

- Add the Boyer-Moore algorithm (skips more characters than KMP in practice).
- Add Aho-Corasick for simultaneous multi-pattern matching.
- Enable custom graph input from a file or random generator.
- Export animation as a GIF for use in presentations.
- Add a side-by-side comparison mode that runs all three algorithms simultaneously on the same input.

REFERENCES:

Books:

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.: Chapter 32: String Matching

[2] Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.: Section 5.3: Substring Search

[3] Knuth, D. E., Morris, J. H., & Pratt, V. R. (1977). Fast pattern matching in strings. SIAM Journal on Computing, 6(2), 323–350.

[4] Rabin, M. O., & Karp, R. M. (1987). Efficient randomized pattern-matching algorithms. IBM Journal of Research and Development, 31(2), 249–260.

Online Resources:

[5] Geeks for Geeks:KMP Algorithm: https://www.geeksforgeeks.org/kmp-algorithm-for-pattern-searching/

[6] GeeksforGeeks: Rabin-Karp: https://www.geeksforgeeks.org/rabin-karp-algorithm-for-pattern-searching/

[7] CP-Algorithms: String Matching: https://cp-algorithms.com/string/prefix-function.html

[8] Oracle Java Docs: https://docs.oracle.com/javase/8/docs/api/javax

YouTube Lectures:

[9] Abdul Bari: KMP Algorithm Explained (String Matching): https://www.youtube.com/watch?v=V5-7GzOfADQ

[10] Abdul Bari: Rabin-Karp String Matching Algorithm: https://www.youtube.com/watch?v=qQ8vS2btsxI

[11] MIT OpenCourseWare: Lecture 9: String Matching (6.006): https://www.youtube.com/watch?v=NinWEPPrkDQ

[12] CS Dojo: KMP Algorithm (Visualized): https://www.youtube.com/watch?v=EL4ZbRF587g

Research Papers:

[13] Boyer, R. S., & Moore, J. S. (1977). A fast string searching algorithm. Communications of the ACM, 20(10), 762–772. https://www.cs.utexas.edu/~moore/publications/fstrpos.pdf

[14] Aho, A. V., & Corasick, M. J. (1975). Efficient string matching: An aid to bibliographic search. Communications of the ACM, 18(6), 333–340. https://cr.yp.to/bib/1975/aho.pdf